Checkpoint 5 — Implementação do Aluno 2

Journal durável, Two-Phase Commit e Incremental State Transfer |

explicação técnica e evidências

GUIA EXPLICATIVO

Fluxos, módulos, funções, decisões, evidências e pendências

Leitura baseada no código e na documentação do repositório

Revisão: 05/11/2026

Uso didático: implementação atual separada de requisitos e trabalho futuro

1. Escopo e módulos

No C5, a parte do Aluno 2 coordena a publicação de ANNOUNCE entre Super Peers com Two-Phase Commit (2PC), registra decisões duráveis em um journal lateral e recupera diferenças de catálogo com Incremental State Transfer (IST).

Os módulos centrais são c4_cluster.c (coordenação e replicação), c4_log.c/c4_log.h (log de operações C4 e journal C5), rpc.c/rpc.h (requisições correlacionadas com prazo), protocol.c/protocol.h (mensagens TCP) e superpeer.c (handlers e integração com o diretório). O teste local do journal está em tests/c5/test_transaction_journal.c; a integração TCP está em tests/c5/transaction_integration.py.

2. Journal .txn e durabilidade

O formato existente de operações C4LG permanece inalterado. A decisão C5 é guardada em um sidecar append-only .txn, com identificador C5TJ, tipo de registro (PREPARE, COMMIT ou ABORT), tamanho, CRC32 e payload. PREPARE inclui a entrada C4LG; COMMIT registra a decisão associada; ABORT registra o TransactionID como tombstone. Cada registro é sincronizado com fsync antes de a operação avançar.

Na inicialização, o journal é validado. Se existir COMMIT durável cuja entrada ainda não foi anexada ao log C4, a recuperação reaplica a entrada antes da reconstrução do catálogo. Reexecução de entrada idêntica é tratada como replay idempotente; conteúdo divergente, corrupção ou truncamento inválido é rejeitado. Um tombstone ABORT impede que PREPARE atrasado reabra a transação.

3. ANNOUNCE coordenado por 2PC

O ANNOUNCE C2 é transformado em uma entrada C4LogEntry com proprietário, bytes do anúncio, índice, versão, TransactionID e CRC. O fluxo evita tornar o anúncio visível antes de haver uma decisão COMMIT.

Fase de preparação

- O coordenador grava PREPARE no journal local.
- Envia M_PREPARE (tipo 10) às réplicas ativas capturadas para a operação, carregando a entrada C4LG codificada.
- Cada réplica valida a origem, o estado ativo e a identidade do coordenador reconhecido; valida também transação, índice/versão e CRC.
- A réplica persiste PREPARE antes de responder voto positivo. Nenhum metadado ANNOUNCE é publicado nessa fase.

Decisão e propagação

Após votos positivos, o coordenador persiste COMMIT. A aplicação da entrada no log C4 e o callback que atualiza o diretório tornam o ANNOUNCE visível. Em seguida, envia M_COMMIT (tipo 11) e aguarda confirmações das réplicas capturadas antes de responder ao Peer de origem.

Se houver voto negativo ou prazo excedido, registra ABORT e envia M_ABORT (tipo 12) aos participantes contatados, inclusive aqueles que ainda não responderam. O handler aceita fases idempotentes; retransmissão de COMMIT reencaminha a decisão para completar réplicas pendentes. Handlers exigem que o remetente seja Super Peer registrado e ativo e que corresponda ao líder conhecido.

4. RPC e correlação de transações

O TransactionID é propagado sem ser substituído entre ANNOUNCE, PREPARE e decisão. O RPC associa resposta à requisição e ao destino, aplica os prazos configurados e evita que respostas atrasadas sejam confundidas com outra fase. Essa correlação permite repetir fases de forma idempotente e identificar a transação que deve ser persistida ou aplicada.

5. Transferência Incremental de Estado (IST)

IST copia apenas o sufixo de log que falta, em vez de enviar o catálogo inteiro. M_IST_REQUEST (25) leva dois campos u64: último índice aplicado e versão. M_IST_BATCH (26) contém contagem u32 e entradas C4LG prefixadas por seus tamanhos u32.

O servidor limita cada resposta ao tamanho máximo de payload do protocolo (5 MiB). O receptor valida limites, CRC, transações, ordem, índice e versão e aplica as entradas faltantes ao log e ao catálogo. O cursor avança a partir do último registro confirmado, permitindo pedir o lote seguinte sem duplicar estado. Se índice/versão indicam gap, a divergência é detectada em vez de ser tratada como estado atualizado.

Sincronização ocorre quando o nó assume coordenação e durante a integração de um seguidor via M_COORDINATOR: o seguidor recupera o sufixo antes de confirmar a sincronização. M_SNAPSHOT e M_STATE_TRANSFER existentes continuam disponíveis para os fluxos anteriores.

6. Testes locais

make test-c5 compila os alvos sob flags estritas e executa test_transaction_journal. O teste aprovado verifica o ciclo de vida e replay do journal, inclusive decisões persistidas e recuperação. O teste separado do Aluno 1, test_cache_lfu, também é executado pelo alvo, mas não é evidência da implementação do Aluno 2.

7. Teste integrado TCP e evidências

O script tests/c5/transaction_integration.py inicializa processos com a configuração tests/config/c5.conf e verifica:

- ANNOUNCE replicado por duas fases e recuperado por um Super Peer atrasado;
- replicação de duas entradas e sincronização incremental do nó atrasado;
- busca/estado do catálogo nos nós participantes;
- ABORT recebido diretamente por TCP, persistência do tombstone e rejeição de PREPARE atrasado;
- paginação IST com cursor, estado atual e detecção de gap;
- interoperabilidade com o Peer e repetição de download byte a byte.

A saída registrada no teste integrado reportou: “2PC/IST TCP: duas ANNOUNCE replicadas e recuperadas pelo Super Peer atrasado; batch, cursor, estado atual e gap OK”. O alvo make test-c5 foi executado com sucesso em 05/11/2026. Essa evidência comprova os cenários escritos no teste, não todas as falhas possíveis.

8. Runner original e runner de integração

tests/c5/run.sh e tests/config/c5.conf foram mantidos intactos, conforme orientação do usuário. O runner original não exercita corretamente a integração C5 nesta árvore: tenta invocar tests/generate_fixtures.sh diretamente, mas o arquivo não tem permissão de execução; além disso, inicia somente Super Peers, sem os Peers necessários ao upload, e contém comandos que suprimem erros. Por isso, o caminho de verificação que passou foi make test-c5 (e a cópia integrada tests/c5/run_integrated.sh), sem alterar o runner ou a configuração originais.

O teste integrado exercita ABORT por TCP e o tombstone, porém não força um voto negativo dentro da chamada de coordenação c4_cluster_submit. IST foi verificado com páginas sequenciais e recuperação efetiva, mas não se forçou um registro grande que exigisse múltiplos lotes de 5 MiB.

9. Limites conhecidos

O líder aplica localmente antes de receber todas as confirmações de réplica; uma interrupção durante a propagação pode deixar logs de réplicas diferentes. O termo de eleição permanece em memória. A suíte cobre falhas por parada e recuperação dos cenários descritos, mas não comprova consistência forte sob partição, consenso/SMR completo, metas globais de disponibilidade, nem desempenho. Essas distinções são importantes ao relatar o estado real do checkpoint.